

Sprint 6 Review Date: Friday May 29th
Scrum Master for Sprint 6: Aarush Kapoor

Section 0 – Sprint 5 Release Readiness Reality Check

You must use observable evidence from:

- Sprint 5 demo behavior
- GitHub activity
- testing behavior
- workflow patterns
- integration stability

Vague answers will not receive full credit.

- What instability, inconsistency, or integration weakness still exists in your system?
Three fragile state patterns from Sprint 5 are still uncorrected on our main branch. (1) Trail Browsing’s filter logic uses a useEffect tied to a derived state slice rather than the raw difficulty/surface type values, so consecutive filter switches leave previously filtered cards on the screen until the program is hard refreshed. (2) The trail detail modal’s outside-click close handler relies on event delegation that misfires when the modal renders above certain trail cards, so closing behavior is inconsistent across cards. (3) The Calculator’s money savings bar inherits the same proportional-scaling helper that caused Sprint 4’s CO2 chart visual ambiguity. None of the three problems has a fix commit on main before we head into Sprint 6.
- Which ONE release risk is most likely to break during Sprint 6?
One release risk that is most likely to break during Sprint 6 is Trail Browsing’s filter logic under repeated use. This is the exact failure mode that broke Sprint 5’s demo on 5/15 and it remains structurally un-refactored at Sprint 6 start. Trail Browsing also sits at the center of Momentum’s planning-tool value proposition, so a repeat of the “works once, fails on the second run” pattern is the most visible failure our audience could see.
- What evidence shows this is still a real risk?
(Must reference demo behavior, testing behavior, GitHub activity, or workflow patterns)
On the Demo on 5/15, two consecutive Trail Browsing runs on the same browser session produced different filter behavior: the first run filtered Easy correctly, the second run left Easy-difficulty cards visible after switching from Easy to Moderate to Hard back to Easy. The trail detail modal also failed to close on outside-click on the third attempted close in the same session. The GitHub commit log shows zero fix commits on main for either issue between 5/15 and Sprint 6 start; the responsible files (TrailBrowse.tsx, TrailDetailModal.tsx) have not been worked on at all.
- What functionality still depends on:
(Check all that apply)
 - ☒ Ideal conditions
 - ☐ Pre-configured state/data
 - ☐ Manual setup steps
 - ☒ Explanation instead of demonstration
 - ☒ Specific execution order

Project 3 – Sprint 6 Planning Document

- ☒ “It works if...” conditions
- ☒ Limited testing coverage
- ☐ Specific team member knowledge
- ☐ None of the above
- What part of your system currently feels MOST fragile?
(Be specific)
Trail Browsing currently feels the MOST fragile (specifically the filter state management in TrailBrowse.tsx and the outside-click close handler in TrailDetailModal.tsx). Both passed a single end-to-end walkthrough on Sprint 5 Dev Day 5 but failed on the second consecutive run during Sprint Review.

Section 1 – Sprint 5 Retrospective Rule Application

- Copy your Sprint 6 rule from the Sprint 5 Retrospective Change Plan (exact wording):
At the start of the final 15 minutes of every class period in Sprint 6 beginning Dev Day 1, Aarush Kapoor (Integration Verifier) will project a clean, logged-out browser on the classroom screen and run every Sprint Goal component back-to-back twice in a single session, and any feature that fails on the second consecutive run, requires verbal explanation, or shows visual inconsistency must be moved back to “In Progress” on GitHub before class ends.
- When EXACTLY does this rule happen during a normal class period?
(Be specific)
The rule happens during the final 15 minutes of every class period in Sprint 6 beginning Dev Day 1.
- What would someone SEE happening if this rule is followed correctly?
Someone would see Aarush opening a fresh incognito browser tab on a different persons computer, log in as a test account, and run every Sprint Goal feature back-to-back twice in the same session.
- Where will this show up in your workflow?
(check all that apply)
 - ☐ Integration work
 - ☒ Testing
 - ☒ Pull requests
 - ☒ Board updates
 - ☐ Daily Scrum
 - ☒ Final validation
 - ☒ Bug fixing
 - ☐ Release preparation
- What is ONE way this rule could still fail under Sprint 6 pressure?
One way this rule could still fail under Sprint 6 pressure is if the walkthrough is not run for consecutive class periods (whether due to development time pressure or absences of team members); this would mean that features would not be thoroughly tested

before being pushed to our final branch for this Sprint (and therefore, bugs might be discovered during the live demo).

- What happens IMMEDIATELY if this rule is not followed?
Aarush runs the git revert command on the offending feature's most recent merge to main during the same class period. The feature's GitHub ticket is reopened with the failure mode documented in the comments. No new merges to main are allowed in that class period anymore until the revert is pushed back. The developer who wrote the reverted feature pairs with Aarush at the next class to write the fix.
- Who enforces this rule?
Aarush Kapoor
- If this person is absent, who takes over enforcement?
Yohaán Ardeni
- How does this rule specifically reduce release or integration risk?
Every feature reaching the Sprint 6 final demo will have passed at least one in-class, two-consecutive run walkthrough on the merged main branch. Anything that fails the second run gets reverted from main the same day, so unstable code physically cannot sit on main during Sprint 6 integration.

Section 2 – Sprint Goal (Integration + Stability)

Sprint 6 is NOT for expansion.

Your Sprint Goal must focus on:

- integration
- stability
- polish
- repeatability
- release readiness
- Sprint Goal
(By the end of Sprint 6, a user will be able to...):
By the end of Sprint 6, a user will be able to repeatedly execute Momentum's full end-to-end flow (log in, browse and filter trails by difficulty and surface type, open and close a trail detail modal, log a ride to the activity calendar, and view CO2 and money savings).
- What existing functionality MUST remain stable?
Every Sprint 1–5 feature already on main must remain STABLE (the nav bar, Supabase authentication (sign in/out, admin vs user), gallery (image and document uploads with per-user persistence), announcements page, calendar/events, Leaflet map with location markers, Trail Browsing functionality, CO2 calculator, money savings display, and the activity log with date validation).
- What would release failure look like during the final presentation?
A repeat of the Sprint 5 Trail Browsing pattern in front of the final-presentation audience (where a feature passes the first demo pass and fails the second) would be release failure during the final presentation. Some examples include stale filter cards visible after consecutive filter switches or a trail detail modal that won't close on

Project 3 – Sprint 6 Planning Document

outside-click. Any of these would replay the exact failure mode the Sprint 5 retrospective documented.

- What specific system behavior must become MORE reliable this Sprint?
Trail Browsing's filter re-rendering on consecutive changes (must produce a clean re-render every time, no stale cards) must become MORE reliable this Sprint. The trail detail modal's outside-click close handler (must close consistently across every trail card in the same session). The Calculator's money savings bar (must scale proportionally without requiring verbal explanation of bar heights).
- What unstable behavior MUST be eliminated before May 29th?
The following unstable behavior must be eliminated before May 29th: filter state staleness on consecutive Trail Browsing filter switches and inconsistent modal close behavior.

Section 3 – Feature Freeze Confirmation

Sprint 6 has a strict Feature Freeze.

No new features may be added.

- What functionality is now considered FROZEN and should NOT change further?
All Sprint 1–5 features currently merged to main are now considered frozen and should not change further: navigation, authentication, gallery (image and document upload), announcements, calendar/events, Leaflet map, Trail Browsing (logic frozen except for bug fixes named in this plan), CO2 calculator, money savings, activity log with date validation.
- What functionality is STILL allowed to change during Sprint 6?
(Examples: bug fixes, UI polish, integration fixes)
Only the three named bug fixes (Trail Browsing filter useEffect, modal outside-click close handler, money savings bar proportional scaling), the new smoke test documentation in /docs/smoke_test.md, demo seed data, CSS polish that does not change DOM structure are allowed to change during Sprint 6.
- What work is EXPLICITLY NOT ALLOWED during Sprint 6?
Any of the 9 open Feature tickets in the GitHub backlog: #6 Environmentally friendly organization of shops, #7 Forum to talk, #8 See Routes on Map, #9 Partnerships option with shops, #10 Favorite Routes, #11 Personalized recs, #12 Gamify app, #13 Showcase type of bike, #15 Customize Routes for fitness levels are NOT ALLOWED during Sprint 6.
- What “tempting” feature or idea is your team intentionally refusing to add? Why?
We are refusing to add Personalized recs (#11) and Gamify app (#12) because neither has previously merged code, and both require significant new state and UI surface, and either would directly compete with the stabilization work the existing Trail Browsing filter and modal handler still need before feature freeze. Both will stay open in the backlog as post-release roadmap items.
- What process will prevent feature creep during Sprint 6?
The following process will prevent feature creep during Sprint 6: (1) All PR descriptions

Project 3 – Sprint 6 Planning Document

must explicitly reference one of the 4 Sprint 6 backlog items by name and PRs without a referenced item will not be reviewed. (2) New GitHub issues opened during Sprint 6 are auto-tagged “Post-release” through our agreement and not worked on in this sprint. (3) Aarush’s two-pass walkthrough script is locked at Sprint 6 Dev Day 1: if a new component or route appears in the running app that is not in the script, the walkthrough flags it as a regression and the merge is reverted.

Section 4 – Scope Lock & Integration Priorities

You may select ONLY work directly related to:

- integration
- stabilization
- regression prevention
- bug fixing
- polish
- release readiness

No expansion work.

- Included Sprint Work (2 – 4 items MAX):

Backlog Item	Why it is REQUIRED before release
Range of Biking Trails	Trail Browsing is Momentum’s most visible planning-tool feature; this is the single highest demo-failure risk going into Sprint 6 which is why it is required before release.
Customize Routes for Fitness Levels	Trail Browsing is Momentum’s most visible planning-tool feature; this is the single highest demo-failure risk going into Sprint 6 which is why it is required before release.
Click or tap here to enter text.	Click or tap here to enter text.
Click or tap here to enter text.	Click or tap here to enter text.

- What work is intentionally NOT included in Sprint 6?
Any of the 9 open Feature tickets (#6, #7, #8, #9, #10, #11, #12, #13, #15) are NOT included in Sprint 6 because they would all require significant new state and UI surface changes.
- What unfinished work is being permanently delayed until after final presentations (if ever)?
All 9 open Feature tickets are permanently delayed past our final presentations: they will remain in the open backlog as a post-release roadmap but have no delivery commitment. Momentum’s MVP is what ships on 5/29; the deferred feature set is future scope only.

- Why is this workload realistic given there is no school on May 22nd, May 25th, and May 26th?
Only 5 dev days remain (5/19 Tue, 5/20 Wed, 5/21 Thu, 5/27 Wed, 5/28 Thu) with the three no-school days (5/22, 5/25, 5/26) splitting the sprint into a 3-day bug-fix block and a 2-day rehearsal block. All three bug-fix tasks (filter, modal, calculator) are owned by people who already wrote the original code in Sprint 5 and are scoped as targeted patches, not rewrites. Dev Day 5 (5/28) is intentionally reserved entirely for walkthrough rehearsals (no new code merges after end of 5/27). The 4-item scope fits the 3-day fix window with one buffer day for missed work.

Section 5 – Integration Risk Analysis

- Which system integration is the HIGHEST technical risk?
The handoff between Trail Browsing and Activity Log. Sprint 5 retro Section 5 flagged this already. Both features share a “selected mode” notion in different state shapes, so any Sprint 6 work that touches Trail Browsing’s filter state could ripple into Activity Log’s ride logging. The filter useEffect refactor in Task 1 is the specific change most likely to expose this dependency.
- What specifically could break when systems combine?
Filter state changes triggering unexpected re-renders in the Activity Log calendar grid (shared global state). The modal close handler fix accidentally breaking the calendar day-click popup (shares modal patterns). The money savings bar CSS polish pushing adjacent layout in the trail card grid (shares the proportional helper). Any of these would surface as a regression on a previously stable feature during Aarush’s daily walkthrough.
- What existing functionality is MOST at risk of regression?
Activity Log persistence (Sprint 4 fragile around state, fixed late in Sprint 4). Trail card rendering on first page load (any filter logic change could disturb it). Calendar day-click popup (shares modal patterns with the trail detail modal we are fixing). Money savings bar in the Calculator (we are changing its proportional scaling).
- What dependency could block your Sprint Goal?
Supabase availability. The entire end-to-end flow depends on Supabase auth, the gallery storage bucket, the rides table, and the locations table. If Supabase has an outage or hits a free-tier rate limit during demo prep, the demo breaks regardless of our code. Mitigation: the team verifies the Supabase connection in the first 2 minutes of every dev day and maintains a local seed dataset.
- What evidence suggests this risk is real?
Sprint 4 retrospective: Activity Log persistence required a manual refresh during demo, which is the original symptom of the shared global state pattern. Sprint 5 retrospective Section 5: the “selected mode” cross-feature dependency is documented clearly. Sprint 4 retrospective: the CO2 bar required verbal explanation, and the same proportional scaling pattern is still in the money savings bar. Every risk listed here has a documented prior failure

Project 3 – Sprint 6 Planning Document

- If this integration fails by Dev Day 3, what scope will be removed immediately?
The Calculator money savings bar polish (Task 3) gets removed first. It is a cosmetic issue, not a functional blocker. If we are still falling behind, the smoke test script (Task 4) gets reduced from a full documented script to a 5-bullet manual checklist. Trail Browsing filter (Task 1) and modal close handler (Task 2) cannot be removed. They are the primary regression risks identified in the SprinSect 5 retrospective

Section 6 – Task System

Tasks MUST:

- be ≤ 1 class period
- include testing
- include regression verification
- be executable by another team member if needed

Add additional rows to this table as necessary by copy/pasting the contents of an existing row and then modifying.

Task	Owner	Related Backlog Item	Done = (clear criteria)	Includes Testing ?	Regression Check
Refactor TrailBrowse.tsx filter useEffect: change dependency array from the derived filteredTrails slice to the raw difficulty and surfaceType state values, so consecutive filter changes always trigger a clean re-render of the trail card list.	Sathkrit h	Trail Browsing Filter Stability	Done = consecutive filter switches in one browser session (Easy, then Moderate, then Hard, then back to Easy) produce a clean re-render every time with no stale cards visible. Verified by Aarush's two-pass walkthrough on the merged main branch.	Yes	Verify the trail card grid still renders correctly on first page load. Verify no console errors on filter change. Verify Activity Log calendar grid is unaffected (shared global state risk per Section 5).
Fix TrailDetailModal.tsx outside-click close handler: replace event delegation with an explicit useRef + useEffect cleanup pattern so	Yohaana	Trail Detail Modal Stability	Done = outside-click closes the modal consistently for at least 5 different trail cards opened in sequence within the same browser session, and the ESC	Yes	Verify the calendar day-click popup (shares modal pattern with TrailDetailModal) still opens and closes

Project 3 – Sprint 6 Planning Document

the modal closes consistently regardless of which trail card opened it.			key also closes the modal, verified by Aarush's two-pass walkthrough.		correctly. Verify no console errors when opening/closing the modal in rapid succession.
Fix Calculator money savings bar proportional scaling: replace the inherited proportional helper with explicit Math.max() normalization so the two bars resize cleanly and labels stay readable for all input distance values without requiring verbal explanation.	Aarush	Calculator Money Savings Bar Polish	Done = bars resize correctly and labels stay readable for input distances of 1, 10, 50, and 100 miles across all four transit modes (bike, car, transit, walking), verified visually in the browser and confirmed in Aarush's walkthrough without any verbal explanation needed.	Yes	Verify the CO2 chart (uses related scaling logic) still renders correctly across the same input range. Verify the Calculator page layout does not overflow on smaller viewports.
Write end-to-end happy-path smoke test script: create /docs/smoke_test.md listing every step Aarush runs during the daily two-pass walkthrough, including expected behavior at each step and known fragile points to watch.	Sriram	End-to-End Happy-Path Smoke Test Script	Done = script is committed to /docs/smoke_test.md, can be run by any team member in under 5 minutes, and covers login, gallery upload, calendar log, calculator, and trail browse with filter and detail modal. Includes explicit second-pass instructions in the same session.	Yes	No code change, so no direct regression risk. Verification: Sathkrith (not the author) runs the script start-to-finish on his own machine and confirms every step works as documented.
Run the first full Sprint 6 baseline two-pass walkthrough on Dev Day 3 (5/21) using	Aarush	Run End-to-End Happy-Path Smoke	Done = walkthrough completes twice consecutively on a clean logged-out browser using a	Yes	This task IS the regression check. It exercises every Sprint 1 to 5

Project 3 – Sprint 6 Planning Document

the new smoke test script. Capture the before-state behavior of all known fragile flows for comparison with later walkthroughs.		Test Script	teammate's computer, results (pass/fail per feature) logged as a comment on the Sprint 6 GitHub board.		feature. Any regression discovered triggers an immediate git revert per the Sprint 6 rule.
---	--	-------------	--	--	--

- What is the SINGLE most critical task in Sprint 6?
Task 1 (Trail Browsing filter refactor, owned by Sath) is the SINGLE most critical task in Sprint 6. It targets the specific code path that caused the Sprint 5 demo failure on 5/15, and Trail Browsing is the most visible piece of Momentum's planning-tool value proposition.
- What would happen if this task failed?
A Sprint 6 demo would replay Sprint 5's exact failure pattern where Trail Browsing works once, fails on the second pass. Aarush's two-pass walkthrough should catch it pre-demo and the escalation in Section 1 would force Trail Browsing out of the demo path entirely, which would minimize the demoable surface to gallery, calendar, and calculator only.
- How will this task continue if the owner is absent?
Yohaán is the secondary owner of this task because he originally implemented the filter logic in Sprint 5 and can pair on the refactor remotely. The smoke test script (Task 4) documents the exact verification steps, so technically any teammate can confirm completion without Sath present.

Section 7 – Regression Prevention Plan

- What previously working functionality MUST be tested repeatedly during Sprint 6?
Every Sprint 1 to 5 feature included in the Sprint Goal: navigation bar, Supabase authentication, gallery (image and document upload), announcements, calendar/events, Leaflet map with markers, Trail Browsing first-pass functionality, CO2 calculator, money savings display, activity log with date validation. All ten exercised every dev day during the two-pass walkthrough.
- When during development will regression testing occur?
In the final 15 minutes of every class period during Aarush's two-pass walkthrough. Plus, an additional 5-minute pre-merge regression check on the changed feature known neighbors (per Section 5 risk map) before any PR is merged to main.
- Who is responsible for regression verification BEFORE work moves to Done?
Aarush Kapoor, as Integration Verifier. A task cannot move from In Progress to Done on the GitHub board until Aarush has run the relevant regression check from the smoke test script on the merged main branch.
- What happens immediately if previously stable functionality breaks?
Aarush runs git revert on the offending merge during the same class period. The

breaking change's GitHub issue is reopened with the failure mode and the affected stable feature documented in the comments. No further merges to main are allowed in that class period. The developer who owned the regression pairs with Aarush at the next class to write the fix.

- How will your team prevent “last-minute stability collapse” before demo day?
No code merges to main after end of class on 5/27 (Dev Day 4). Dev Day 5 (5/28) is reserved entirely for repeated walkthrough rehearsals and presentation prep. Any bug surfaced on 5/28 that cannot be fixed and re-walked-through in under 15 minutes is treated as “demo around it.” The affected feature gets removed from the demo path instead of patched under time pressure.
- What functionality will be tested MULTIPLE TIMES before the Sprint 6 Review?
Every feature in the Sprint Goal will be tested twice per day inside Aarush's walkthrough. With 5 dev days, that is at least 10 separate runs per feature before Sprint Review on 5/29 (8 if Dev Day 1 is consumed by planning). Trail Browsing's filter logic and the trail detail modal will receive extra scrutiny on Dev Days 3, 4, and 5 because they are the named Sprint 6 fix targets.

Section 8 – Testing & Final Validation

- What testing must happen DAILY during Sprint 6?
Aarush's two-pass walkthrough using the smoke test script (final 15 minutes of class). Per-task testing on completion before any task moves to Done. A 2 minute Supabase connection check in the first 2 minutes of every dev day (per Section 5 dependency mitigation).
- What testing must happen BEFORE any PR merge?
(1) The PR author runs the smoke test script end-to-end on their feature branch and pastes the result in the PR description. (2) A teammate who is not the PR author opens the branch and runs the same script independently. (3) Both PR description and reviewer comment must explicitly state which Sprint 6 backlog item this PR addresses. PR is only mergeable after both checks pass and the backlog reference is present.
- What final validation steps must happen BEFORE May 29th?
Dev Day 4 (5/27): full demo rehearsal #1 with all Sprint Goal features running twice consecutively under Aarush's walkthrough; demo timing measured against the 10-minute presentation window. Dev Day 5 (5/28): final dress rehearsal under presentation conditions (projector, audio, demo account, the same room layout planned for 5/29). No merges to main allowed on 5/28.
- How will your team verify the demo works repeatedly and not just once?
The walkthrough rule structurally enforces it every dev day, two consecutive runs in the same browser session, on a different team member's computer than the previous walkthrough. Any feature that passes one run and fails against the other is treated as broken and reverted. By 5/29, every Sprint Goal feature will have passed at least 8 two-pass walkthroughs across the 5 dev days.

Project 3 – Sprint 6 Planning Document

- What would someone SEE if your release process is functioning correctly?
A GitHub board where every Done task has a corresponding walkthrough confirmation comment from Aarush. PR descriptions that explicitly reference a Sprint 6 backlog item and contain pasted smoke test results. Aarush at the classroom projector during the final 15 minutes of every dev day. Zero “let me just refresh” moments during dress rehearsal on 5/28. A demo on 5/29 with zero verbal explanations of fragile behavior.

Section 9 – Workflow Discipline & Release Protection

- When EXACTLY is the board updated each class?
Either immediately after a task is completed, or 10 minutes to class. If nothing has been moved to either in progress or done by the end of class, the Scrum master must reevaluate if it is broken down and shift all attention to it to ensure completion.
- Who verifies that the board reflects reality?
Aarush
- What rule prevents unfinished work from being labeled Done?
All work that must be moved to done must be verified as functional by another member of the team. This will happen as we ask everyone to first push their code after they believe it is done, and then have someone verify the features. Only once it has been pushed to the main branch is it considered complete.
- What rule prevents “silent feature creep” during Sprint 6?
All PR descriptions must explicitly reference one of the 4 named Sprint 6 backlog items by name. PRs without a reference will not be reviewed. New GitHub issues opened during Sprint 6 are auto-tagged “Post-release” by a team agreement and not worked on. Aarush’s walkthrough script is locked at Sprint 6 Dev Day . Any new component or route appearing in the running app that is not in the script is flagged as a regression and reverted.
- How will workflow discipline remain stable during schedule compression?
By compressing what we commit to, not what we enforce. 4 backlog items, 5 dev days, single class period tasks owned by the people who wrote the original code. The walkthrough rule and the merge-stop on 5/27 are structural, not optional. The team commits to running the full walkthrough every dev day even if it means leaving 5 minutes of development time on the table.
- What workflow behavior would immediately signal that the Sprint is failing?
A PR description without a referenced Sprint 6 backlog item. A merge to main after end of class on 5/27. A walkthrough that gets skipped, postponed, or moved after class. A new GitHub issue opened during Sprint 6 without the Post-release tag. Any of these means the rule is breaking, and Aarush calls an emergency scope meeting that evening through a discord call.

Section 10 – Compressed Timeline Adjustment Plan

Project 3 – Sprint 6 Planning Document

- Given the reduced instructional time, what work becomes unrealistic?
Any of the 9 open Feature tickets. Any new Supabase table. Any refactor larger than two file. Any rewrite of the fragile code. Test infrastructure beyond the smoke test script. Unrelated technical debt cleanup. UI polish beyond the named money savings bar fix.
- What MUST be complete BEFORE the no-school days begin?
All three bug fixes (Trail Browsing filter, modal close handler, money savings bar). Smoke test script committed to /docs/smoke_test.md. First full two-pass walkthrough run by Aarush with the new code on Dev Day 3 (5/21). This makes 5/19, 5/20, 5/21 the bug-fix block, ending before the 5/22–5/26 no-school stretch.
- What MUST be complete BEFORE May 29th demo day?
Two completed demo rehearsals (Dev Day 4 on 5/27 and Dev Day 5 on 5/28). Zero open critical bugs on the Sprint 6 backlog items. Zero features in the Sprint Goal that have failed any walkthrough in the last 2 dev days. Final dress rehearsal under presentation conditions completed on 5/28 with zero verbal explanations needed.
- If your Sprint falls behind by Dev Day 2, what gets removed FIRST?
The Calculator money savings bar polish (Task 3, owned by Aarush) is removed first. It is a cosmetic carryover from Sprint 4 and not a functional blocker. Removing it frees Aarush's coding time to support Trail Browsing filter or modal fixes if those are slipping. If the sprint is still behind by Dev Day 3 (5/21), the smoke test script is reduced from a full documented file to a bullet point checklist

Section 11 – Final Presentation Readiness

- What still prevents your system from feeling “final-release ready”?
Trail Browsing's filter logic not yet refactored on main. Trail detail modal's outside-click close handler is still inconsistent. Money savings bar still inherits the Sprint 4 proportional scaling pattern that requires verbal explanation. No documented smoke test script yet exists.
- What user experience issues still require polish?
Trail Browsing consecutive filter changes still show stale cards that are place holders from the prior sprint. Trail detail modal does not reliably close on outside click across all trail cards. Money savings bar proportional scaling causes visual ambiguity at low distance values (example: 1 mile). No smooth animation or loading state between filter changes (lowest priority).
- What part of the demo still feels weakest?
Trail Browsing on the second consecutive run. This is the specific failure mode from Sprint 5 Sprint Review on 5/15 and is structurally uncorrected at Sprint 6 start. Until Task 1 ships and passes two consecutive walkthroughs, this remains the weakest demo segment.
- What part of the system still depends too heavily on explanation?
The money savings bar in the Calculator (and the CO2 chart it inherited from). At Sprint 4 demo, the team had to verbally explain bar proportions. Sprint 5 retrospective Section 5 confirms this pattern is still present in the money savings bar. Until Task 3 ships, the

demo still depends on saying “notice how the bike bar is shorter than the car bar” rather than the visual speaking for itself.

- What must be true by May 29th for your team to feel ready for final presentations?
Every Sprint Goal feature must pass Aarush’s two-pass walkthrough on at least two different class periods before the demo for us to feel ready for the final presentation. The smoke test script must run cleanly twice in a row under presentation conditions on 5/28. We should be able to hand the demo computer to a stranger and the flow should still work.

Section 12 – Failure Prevention & Scope Reduction Plan

- Earliest warning sign your Sprint is failing:
If a walkthrough is skipped or postponed on any dev day in the bug-fix block (5/19, 5/20, 5/21).
- If [X] is not complete by [time], we will [action]
If Trail Browsing filter refactor (Task 1) is not merged and walkthrough-verified by the end of class on 5/21 (Dev Day 3, last day before the no-school stretch), Aarush (I) will call an emergency scope meeting that evening and cut the Calculator money savings bar polish (Task 3) from Sprint 6 scope to reallocate the team's time to the filter fix on 5/27.
- Who makes the scope reduction decision?
Aarush as Sprint 6 Scrum Master and Integration Verifier, with Sathkrith’s input as backup enforcer. Either may call the emergency meeting; both must agree on what gets cut. If they disagree, the default decision is to cut the lowest-priority item (Calculator polish first, then smoke test script automation).
- What functionality will be removed FIRST if needed?
Calculator money savings bar polish (Task 3) is cut first — cosmetic, lowest user impact, and removing it frees Aarush’s development time for the higher-priority Trail Browsing fixes. Second cut: smoke test script reduced from a full /docs/smoke_test.md file to a 5-bullet manual checklist Aarush keeps in his notebook.
- What functionality CANNOT fail under any circumstances?
Login/authentication. Navigation between pages. Trail Browsing first-pass functionality (filtering once cleanly). Activity log persistence. Without these four the demo has no spine. Trail Browsing’s second-pass repeatability would be cut from the demo before any of these four is allowed to fail.

Section 13 – Final Commitment Checklist

- ☒ Our Sprint Goal focuses on integration and stability — not expansion
- ☒ We are following Feature Freeze
- ☒ We identified realistic integration risks
- ☒ We identified regression risks
- ☒ Every task includes testing
- ☒ Every task includes regression verification

Project 3 – Sprint 6 Planning Document

- ☒ Our scope reflects the compressed timeline
- ☒ We have a clear plan to reduce scope early if needed
- ☒ Our workflow discipline expectations are clear
- ☒ Our board will reflect reality throughout the Sprint
- ☒ We are prioritizing stability over additional features
- ☒ We understand that working once is not success